

Informe de Normalización – Sistema de Ventas

Curso: Bases de Datos Relacionales con PostgreSQL

Módulo: 2 – Normalización

Empresa: Ventas Express S.A.S.

1. Diagnóstico de la tabla cruda

1.1 Atributos no atómicos (incumplen 1FN)

Los siguientes campos contienen múltiples valores separados por comas en una sola celda:

Campo	Ejemplo del problema
productos_codigos	'P001,P002' → dos productos en una celda
productos_nombres	'Mouse USB,Teclado mecanico'
categorias	'Perifericos,Perifericos'
cantidades	'2,1'
precios_unitarios	'45000,180000'
descuentos	'0,0'

Estos campos violan la **Primera Forma Normal** porque una celda debe contener un único valor atómico (indivisible).

1.2 Datos repetidos (redundancia)

Dato repetido	Por qué es un problema
cliente_nombre, cliente_email, cliente_telefono, cliente_direccion, cliente_ciudad	Se repiten en cada venta del mismo cliente (CC101 aparece en V1001 y V1003)
vendedor_nombre, vendedor_zona	Se repiten en cada venta del mismo vendedor
categoria	Se repite para cada producto que pertenece a la misma categoría

1.3 Anomalías identificadas

Anomalía de inserción:

No se puede registrar un producto nuevo en el sistema sin asociarlo a una venta. Si se quiere agregar el producto "Audífonos Bluetooth" al catálogo antes de que se venda, no hay forma de hacerlo en la tabla cruda.

Anomalía de actualización:

Si Maria Gomez cambia su teléfono, hay que actualizar TODAS las filas donde aparece CC101. Si se actualiza

solo una fila, la base de datos queda inconsistente con datos distintos para el mismo cliente.

Anomalia de eliminación:

Si se elimina la venta V1002 (la única venta de Juan Perez), se pierde toda la información del cliente Juan Perez, aunque quizás se quería conservar su registro para futuros contactos.

1.4 Dependencias funcionales identificadas

Dependencia funcional	Tipo
cliente_doc → cliente_nombre, cliente_email, cliente_telefono, cliente_direccion, cliente_ciudad	Total (el doc identifica al cliente)
vendedor_id → vendedor_nombre, vendedor_zona	Total (el id identifica al vendedor)
producto_codigo → producto_nombre, categoria	Total (el código identifica al producto)
categoria → (nombre de categoría)	Transitiva a través del producto
venta_id → fecha_venta, cliente_doc, vendedor_id, metodo_pago	Total (la venta tiene su propia identidad)
venta_id + producto_codigo → cantidad, precio_unitario, descuento	Compuesta (detalle de cada línea)

2. Primera Forma Normal (1FN)

Objetivo: Eliminar grupos repetitivos y garantizar que cada celda tenga un único valor atómico.

Transformación: Cada fila que tenía múltiples productos separados por comas se descompone en múltiples filas, una por cada producto.

Ejemplo:

Antes (tabla cruda):

V1001		CC101		'P001,P002'		'2,1'		'45000,180000'
-------	--	-------	--	-------------	--	-------	--	----------------

Después (1FN):

V1001		CC101		P001		2		45000
V1001		CC101		P002		1		180000

Clave compuesta provisional en 1FN: (venta_id, producto_codigo)

Esta clave garantiza unicidad: una venta puede tener cada producto solo una vez en su detalle.

Problema que persiste después de 1FN: Los datos de cliente y vendedor siguen duplicados en cada fila, porque aún dependen solo de `venta_id` o de `cliente_doc`, no de la clave compuesta completa.

3. Segunda Forma Normal (2FN)

Objetivo: Eliminar dependencias parciales. Un atributo que depende solo de *parte* de la clave compuesta debe moverse a su propia tabla.

La clave compuesta en 1FN es: (`venta_id`, `producto_codigo`)

Dependencias parciales encontradas:

Atributo	Depende de...	Acción
<code>cliente_nombre</code> , <code>cliente_email</code> , <code>cliente_telefono</code> , <code>cliente_direccion</code> , <code>cliente_ciudad</code>	Solo de <code>cliente_doc</code> (que a su vez está en la venta)	→ Tabla <code>clientes</code>
<code>vendedor_nombre</code> , <code>vendedor_zona</code>	Solo de <code>vendedor_id</code>	→ Tabla <code>vendedores</code>
<code>producto_nombre</code> , <code>categoria</code>	Solo de <code>producto_codigo</code>	→ Tabla <code>productos</code>
<code>fecha_venta</code> , <code>metodo_pago</code> , <code>entidad_pago</code>	Solo de <code>venta_id</code>	→ Tabla <code>ventas</code>
<code>cantidad</code> , <code>precio_unitario</code> , <code>descuento</code>	De <code>venta_id</code> + <code>producto_codigo</code> (clave completa)	→ Tabla <code>detalle_venta</code>

Entidades resultantes después de 2FN:

- `clientes`
- `vendedores`
- `productos`
- `ventas`
- `detalle_venta`

4. Tercera Forma Normal (3FN)

Objetivo: Eliminar dependencias transitivas. Un atributo no debe depender de otro atributo que no sea la clave primaria.

Dependencias transitivas encontradas:

Dependencia transitiva	Problema	Acción
<code>producto_codigo</code> → <code>categoria_nombre</code>	<code>categoria_nombre</code> depende del producto, no directamente de ninguna clave de venta	→ Tabla <code>categorias</code> separada

Dependencia transitiva	Problema	Acción
venta_id → metodo_pago → entidad_pago	entidad_pago depende de metodo_pago, no directamente de la venta	→ Tabla metodos_pago separada

Razonamiento:

- Si se cambia el nombre de la categoría "Perifericos" a "Periféricos" (con tilde), habría que actualizar cada producto. Con una tabla **categorias** independiente, se actualiza en un solo lugar.
- **entidad_pago** (Bancolombia, Visa) depende del método de pago, no de la venta en sí. Con una tabla **metodos_pago** se evita que el mismo método se escriba diferente en distintas ventas.

Decisión sobre **total_venta**:

No se almacena. Es un dato calculable desde **detalle_venta** con la fórmula:

$SUM(cantidad * precio_unitario - descuento)$

Guardarlo generaría inconsistencias si los datos del detalle cambian.

Decisión sobre **precio_unitario** en **detalle_venta**:

Se conserva en el detalle (no se toma del catálogo). Esto respeta la regla de negocio 6: el precio de un producto puede cambiar en el futuro, pero el precio histórico de una venta pasada debe mantenerse.

5. Modelo Final – Tablas resultantes

clientes

PK	Atributo	Tipo
✓ PK	cliente_doc	VARCHAR(20)
	cliente_nombre	VARCHAR(100) NOT NULL
	cliente_email	VARCHAR(120) UNIQUE
	cliente_telefono	VARCHAR(30)
	cliente_direccion	VARCHAR(150)
	cliente_ciudad	VARCHAR(80)

vendedores

PK	Atributo	Tipo
✓ PK	vendedor_id	VARCHAR(10)
	vendedor_nombre	VARCHAR(100) NOT NULL
	vendedor_zona	VARCHAR(80)

categorias

PK	Atributo	Tipo
✓ PK	categoria_id	SERIAL
	categoria_nombre	VARCHAR(80) UNIQUE NOT NULL

productos

PK/FK	Atributo	Tipo
✓ PK	producto_codigo	VARCHAR(10)
	producto_nombre	VARCHAR(100) NOT NULL
	precio_catalogo	NUMERIC(12,2) CHECK >= 0
🔗 FK	categoria_id	INT → categorias

metodos_pago

PK	Atributo	Tipo
✓ PK	metodo_id	SERIAL
	metodo_nombre	VARCHAR(80) UNIQUE NOT NULL
	entidad_pago	VARCHAR(80)

ventas

PK/FK	Atributo	Tipo
✓ PK	venta_id	VARCHAR(10)
	fecha_venta	DATE NOT NULL
🔗 FK	cliente_doc	VARCHAR(20) → clientes
🔗 FK	vendedor_id	VARCHAR(10) → vendedores
🔗 FK	metodo_id	INT → metodos_pago

detalle_venta

PK/FK	Atributo	Tipo
✓ PK	venta_id	VARCHAR(10) → ventas
✓ PK	producto_codigo	VARCHAR(10) → productos
	cantidad	INT CHECK > 0
	precio_unitario	NUMERIC(12,2) CHECK >= 0
	descuento	NUMERIC(12,2) DEFAULT 0 CHECK >= 0

6. Verificación del checklist final

- ☒ La tabla final no contiene productos_codigos, cantidades o precios separados por comas.
- ☒ Cada tabla tiene una clave primaria clara.
- ☒ Las relaciones tienen claves foráneas activas.
- ☒ Las cantidades no aceptan valores menores o iguales a cero (CHECK cantidad > 0).
- ☒ Los productos no se duplican innecesariamente.
- ☒ Los clientes no se duplican por cada compra.
- ☒ El total de la venta se puede calcular con SQL desde detalle_venta.
- ☒ El diagrama ER coincide con los scripts SQL entregados.
- ☒ El README explica cómo ejecutar los scripts.
- ☒ Las consultas de validación se ejecutan sin errores.